

SEMINAR

XÂY DỰNG PHẦN MỀM DÙNG GENAI

NHÓM SINH VIÊN THỰC HIỆN

NGUYỄN HỮU ANH KHOA

NGUYỄN LÊ QUỲNH HƯƠNG

VÕ THÀNH DANH

TRANG GIA HUY

FROM CODE TO PRODUCTION WITH GENAI

01

DOCUMENTING CODE WITH GENAI

02

WRITING AND MAINTAINING UNIT TESTS

03

GENAI FOR RUNTIME AND MEMORY
MANAGEMENT

04

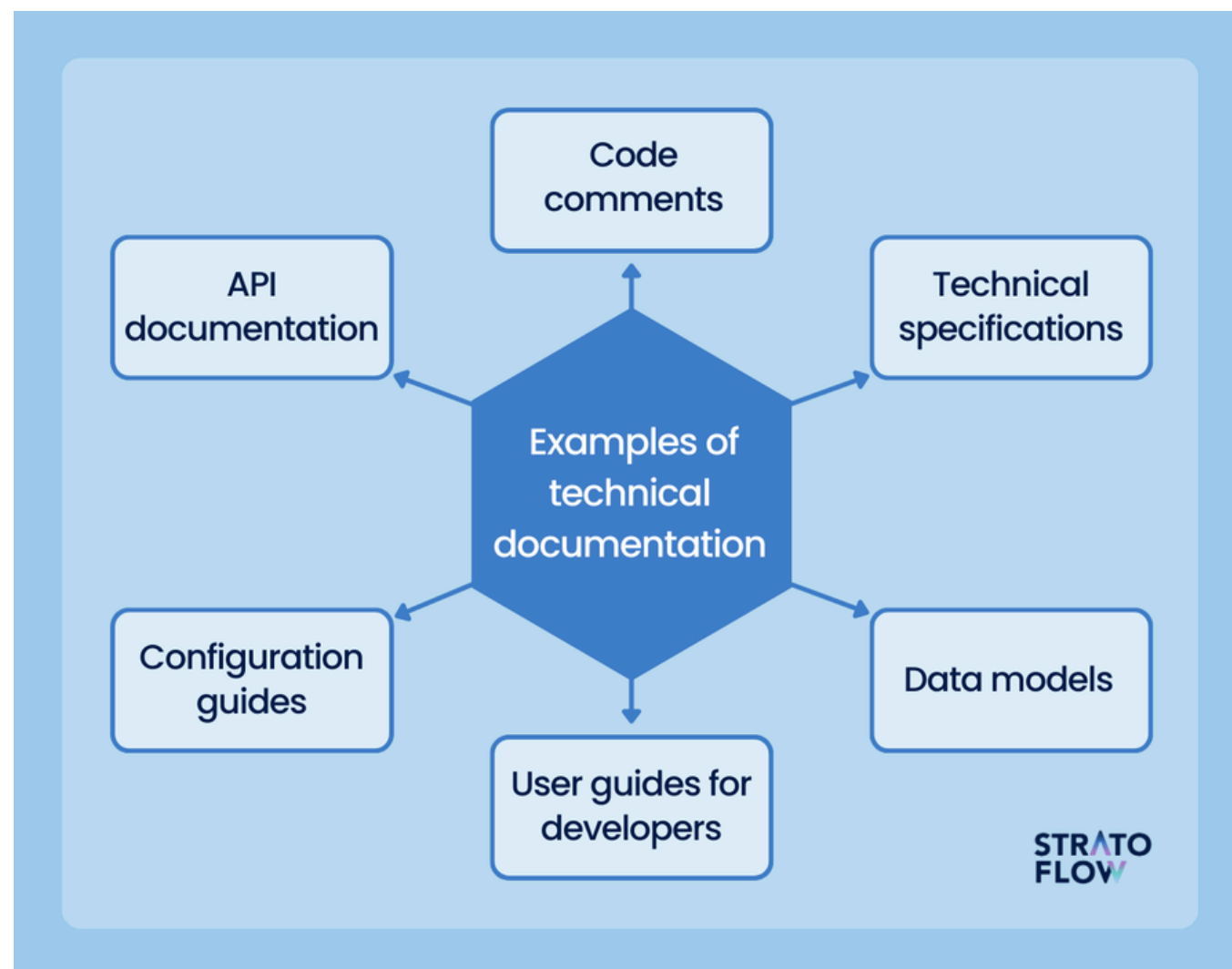
GOING LIVE WITH GENAI: LOGGING,
MONITORING, AND ERRORS

05

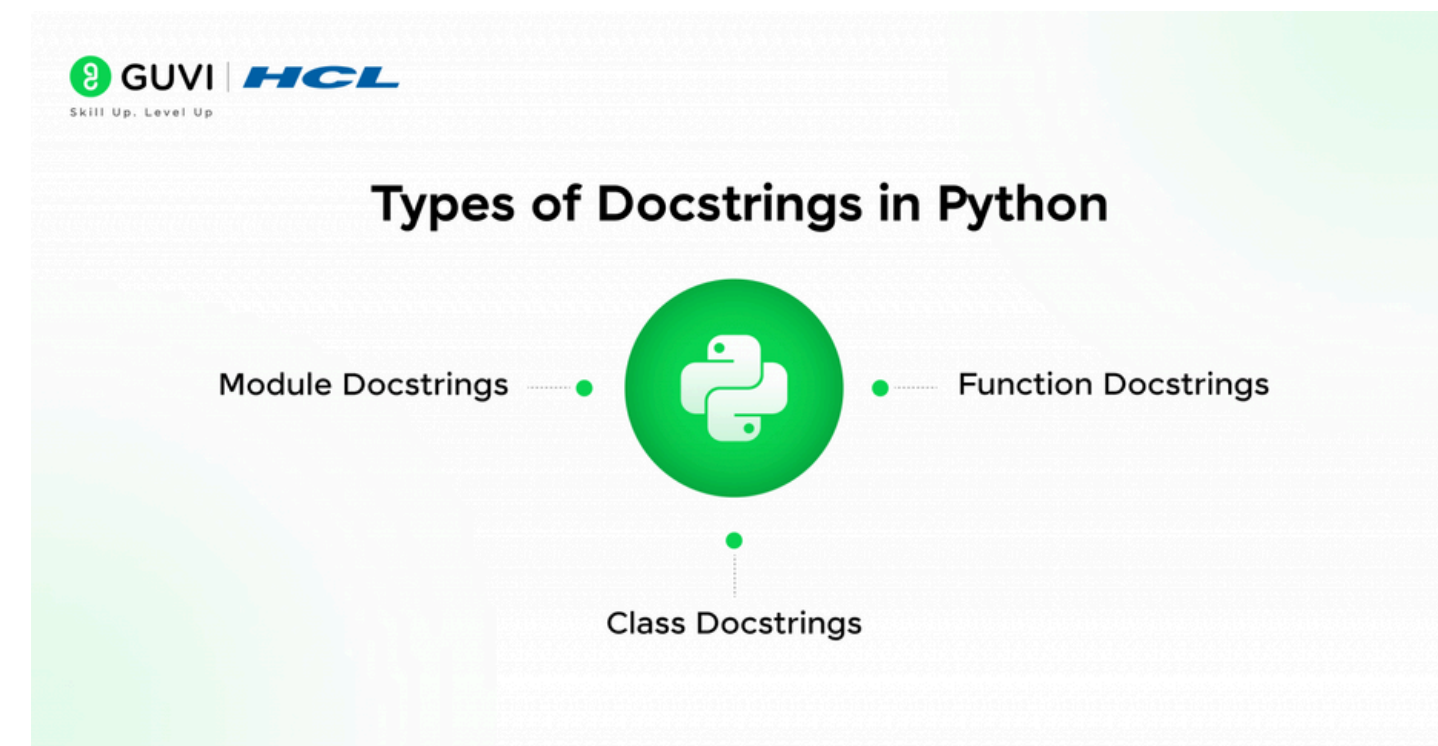
ARCHITECTURE, DESIGN, AND THE FUTURE

DOCUMENTING CODE WITH GENAI

1. SOFTWARE DOCUMENT



2. DOCSTRINGS TRONG PYTHON



DOCUMENTING CODE WITH GENAI

3. SỬ DỤNG GENAI ĐỂ TẠO DOCSTRINGS

GITHUB COPILOT Contextual Autocomplete, Feature-based Generation, Slash Commands

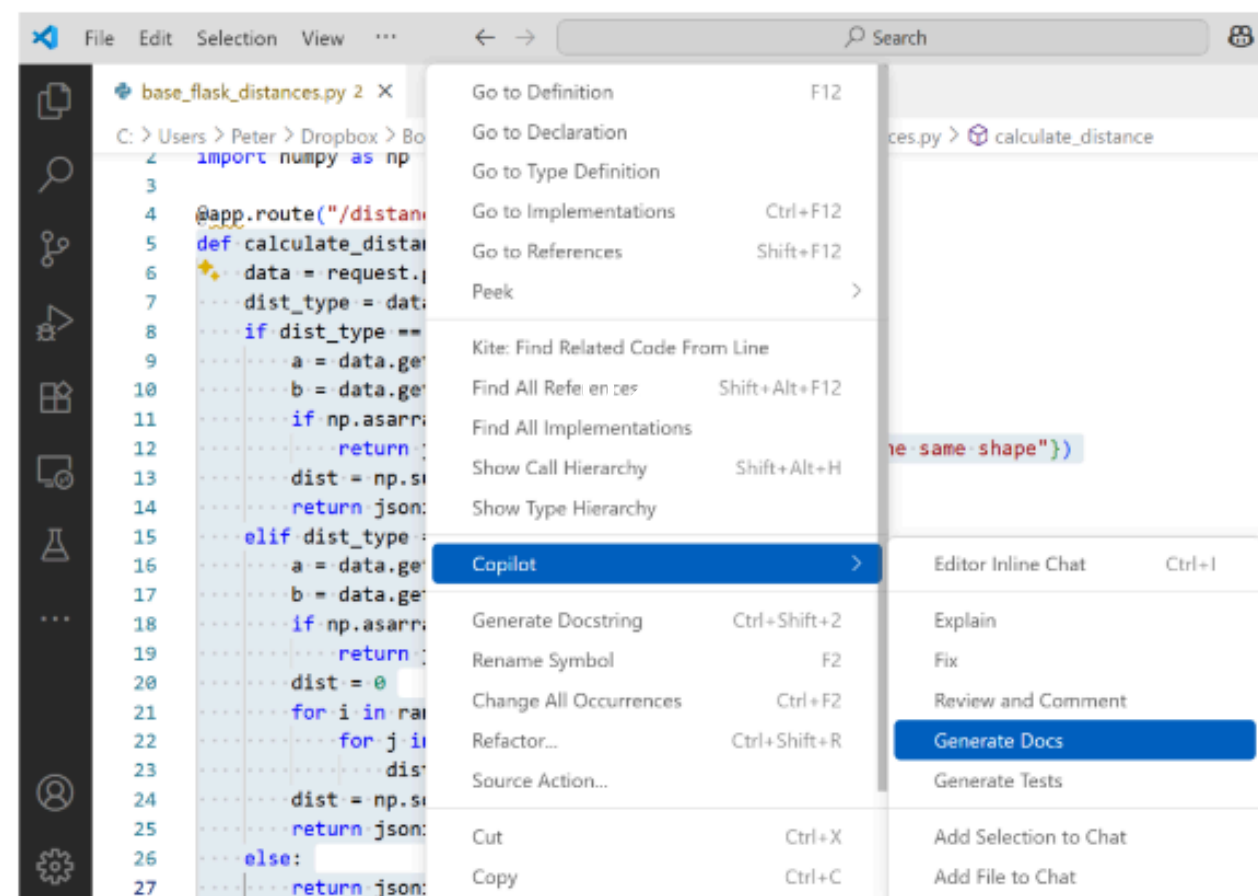


Figure 12.1: By highlighting the method and right-clicking, Copilot generates a docstring

```
Calculate distance between two matrices using L1 or L2 metrics
blank line contains whitespace
Accepts POST requests with JSON payloads containing two matrices
of float type. Supports both Manhattan (L1) and Euclidean (L2) distance metrics.
blank line contains whitespace
Expected JSON payload:

{
  "df1": [float, ...] or [[float, ...], ...], # First matrix
  "df2": [float, ...] or [[float, ...], ...], # Second matrix
  "distance": "L1" or "L2"                  # Distance metric
}

blank line contains whitespace
Returns: flask.Response: JSON response containing either:
- {"distance": <float>} if calculation is successful.
- {"error": <str>} if inputs are invalid or shapes don't match
blank line contains whitespace
No exceptions raised. All errors are returned as JSON error responses.
```

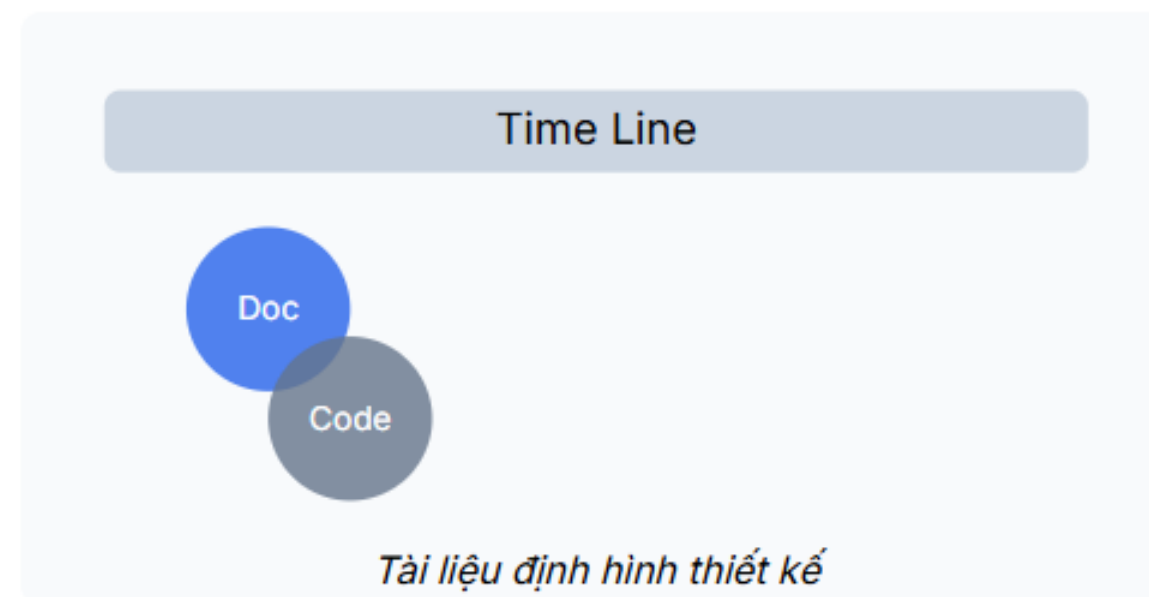
DOCUMENTING CODE WITH GENAI

4. MỐI QUAN HỆ GIỮA CODE VÀ DOCSTRING

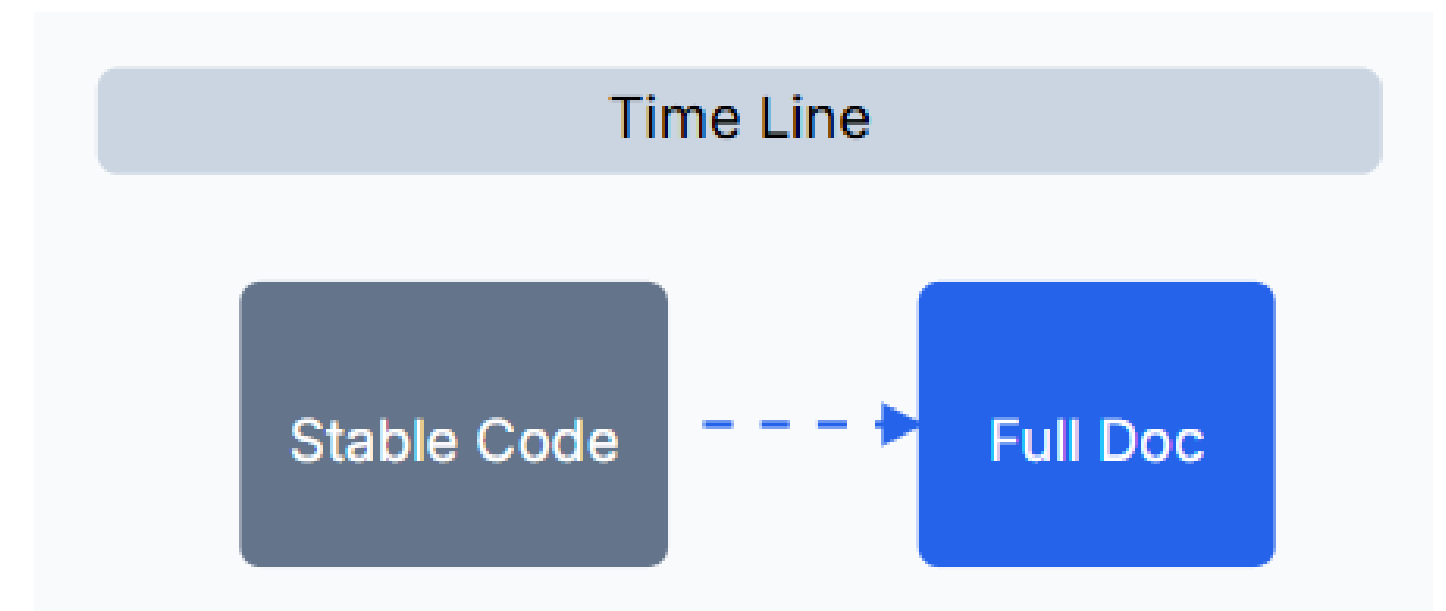
Docstring lỗi thời (Outdated)

Hướng tiếp cận

Code-first Documentation



Post-development Documentation



DOCUMENTING CODE WITH GENAI

5. GIẢI PHÁP VỚI GENAI

Tự động cập nhật docstring

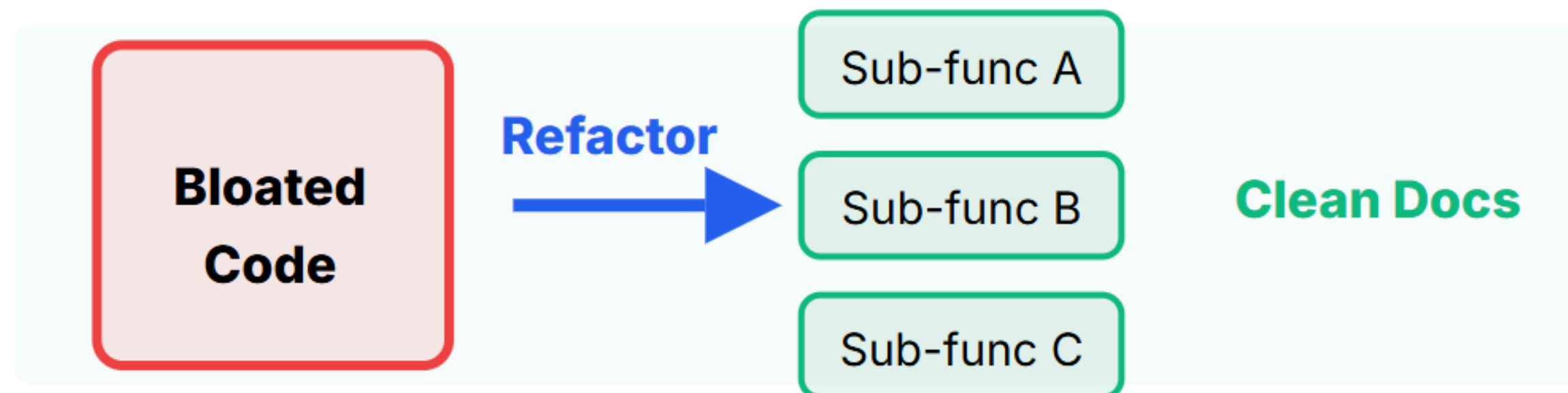
Phát hiện docstring không khớp

Sử dụng Github Copilot

Sử dụng OpenAI API

```
z Supercharged-Coding-with-Gen-AI
a # 23 ~7
python -m ch12.openai_docstring_review
Function: calculate_distance
Docstring: Calculate distance between two arrays from a POST request.
Source: @app.route("/distances", methods=["POST"])
def calculate_distance():
    """Calculate distance between two arrays from a POST request."""
    a, b, dist_type = parse_request_parameters(request)
    dist_func = {"L1": get_manhattan_dist, "L2": get_euclidean_dist}.get(dist_type)
    dist = dist_func(a, b)
    return jsonify({"distance": dist})
```

6. REFACTORING VÀ DOCSTRINGS



Kiểm tra giúp tôi trong source code có docstring nào bị lỗi thời không

Reviewed 4 files and checked lines 1 to 60

Kiểm tra khám phá các lỗi docstring bị lỗi thời:

Tôi tìm thấy 3 lỗi docstring không khớp với implementation:

× Lỗi 1: ch12/distances.py dòng 31

Vấn đề: Docstring nói "squared" nhưng code lại lấy square root.
Sửa: Docstring phải là "Return the Euclidean (L2) distance" (bỏ "squared")

WRITING AND MAINTAINING UNIT TESTS

1. UNIT TESTING VÀ VAI TRÒ TRONG PHÁT TRIỂN PHẦN MỀM

ĐẶC ĐIỂM CỐT LÕI:

Tính khu biệt (Atomic)

Tính độc lập (Isolation)

Khả năng tự động hóa (Automation)

VAI TRÒ CỦA UNIT TEST

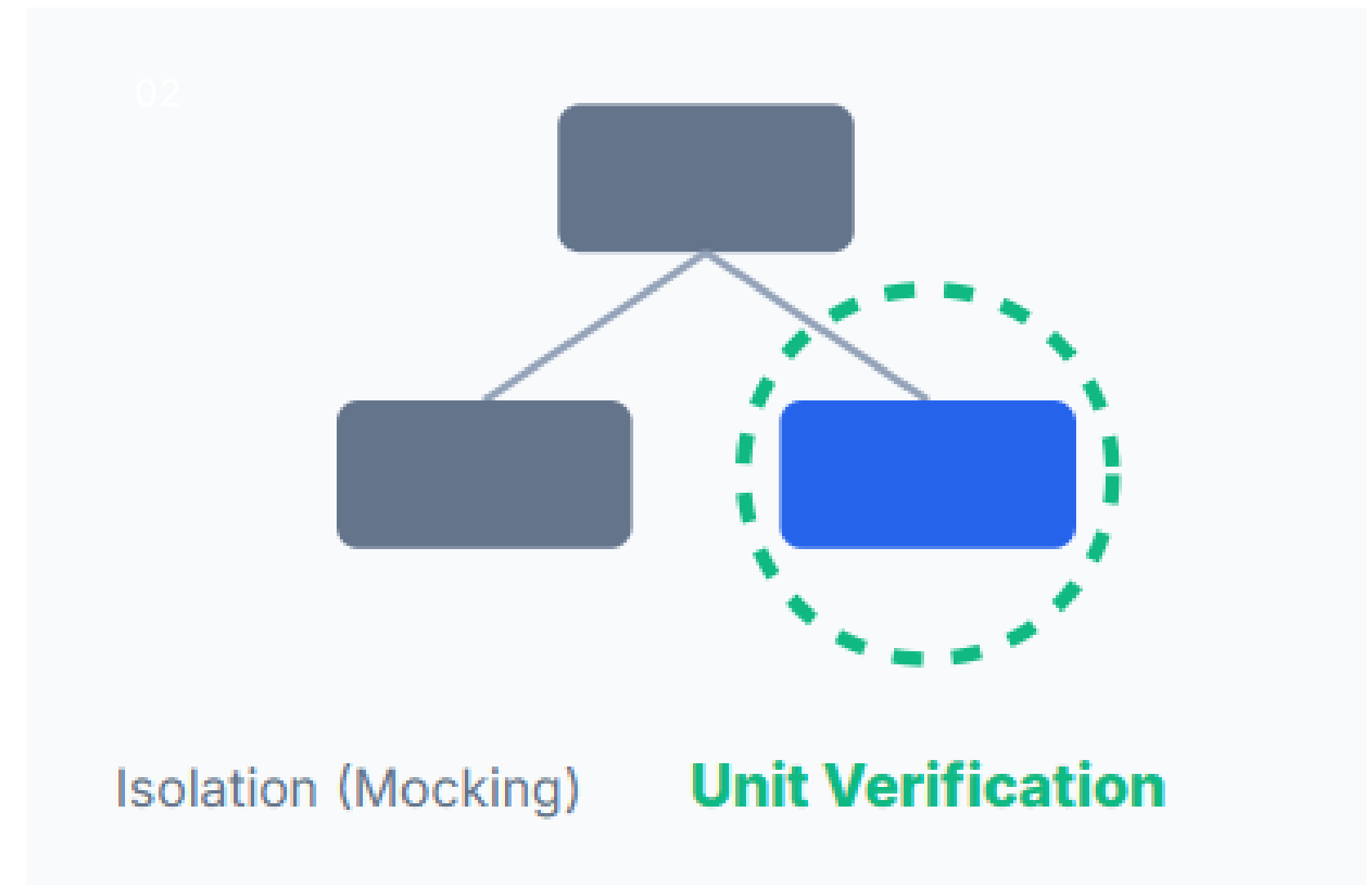
Phát hiện lỗi sớm (Early Bug Detection)

Hỗ trợ tái cấu trúc mã nguồn an toàn (Safe Refactoring)

Xác minh yêu cầu nghiệp vụ

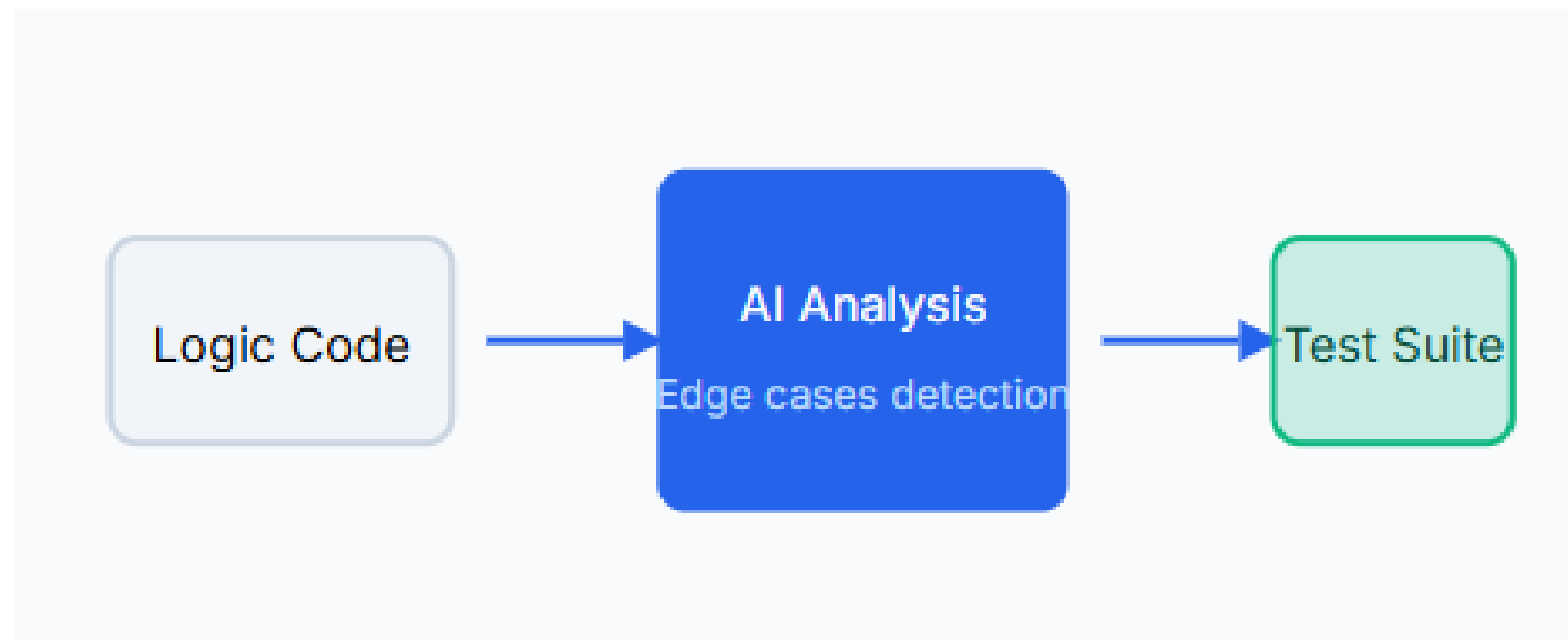
Tài liệu mô tả hành vi hệ thống

Lớp bảo vệ chống lỗi hồi quy (Regression Guard)



BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

2. ỨNG DỤNG GENAI TRONG UNIT TESTING

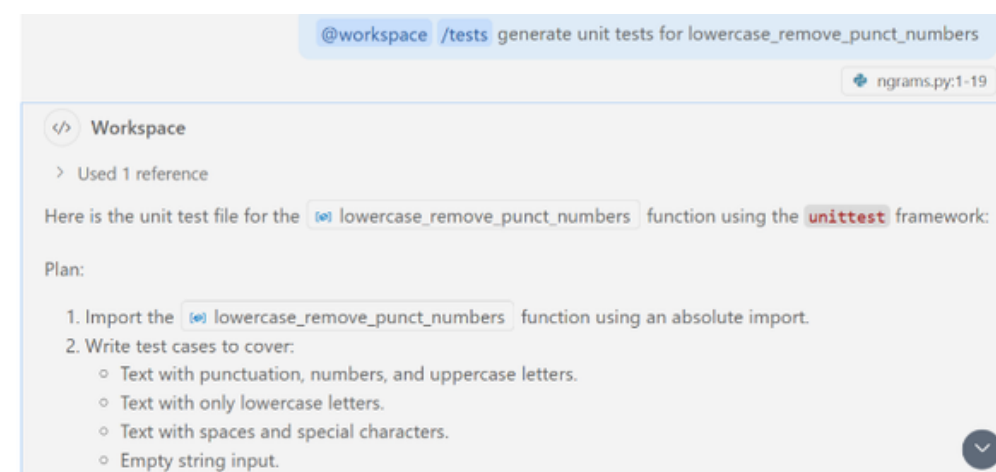
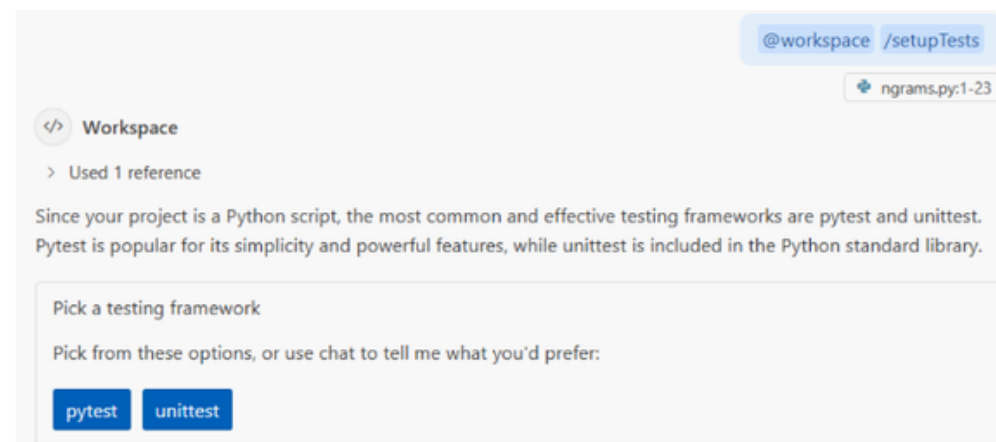


```
You, 2 days ago | 1 author (You)
1 import re
2
3 def lowercase_remove_punct_numbers(text, supercharte=True):
4     return re.sub(r'^a-z\s', '', text.lower())
5
6 def multiple_to_single_spaces(text):
7     letters_single_spaces = re.sub(r'\s+', ' ', text)
8     return letters_single_spaces
9
10 def create_ngrams(text, n) -> list:
11     '''create a list of n-gram tuples from the input text.'''
12     processed_text = lowercase_remove_punct_numbers(text)
13     single_space_processed = multiple_to_single_spaces(processed_text)
14     u = [single_space_processed[i:i+n] for i in range(len(single_space_processed)-n+1)]
15     return u
16
17 if __name__ == "__main__":
18     text = "This is a sample text $ABC% for creating n-grams."
19     n = 3
20     print(create_ngrams(text, n))
```

```
@ ~/Projects/Seminar-Vibe-Coding/Tuan06/Supercharged-Coding-with-Gen-AI :: khoa 8 23 ~7 | Tuan06 | x 100 x | 16:16:10
→ python -m ch13.ngrams.ngrams
The output of this function is a list of 3-grams that span the text input string:
['thi', 'his', 'is ', 's i', 'is ', 's a', 'a ', 'a s', 'sa', 'sam', 'amp', 'mpl', 'ple', 'le ', 'e t', 'te', 'tex', 'ext', 'xt ', 't a', 'ab', 'abc',
'bc ', 'c f', 'fo', 'for', 'or ', 'r c', 'cr', 'cre', 'rea', 'eat', 'ati', 'tin', 'ing', 'ng', 'g n', 'ng', 'ngr', 'gra', 'ram', 'ams']
```

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

2. ỨNG DỤNG GENAI TRONG UNIT TESTING SỬ DỤNG GITHUB COPILOT VÀ CHAT GPT



```
You, 2 days ago | 1 author (You)
1 import unittest
2 from src.ngrams import lowercase_remove_punct_numbers
3
4 # File: test_ngrams.py
5
6 You, 2 days ago | 1 author (You)
7 class TestLowercaseRemovePunctNumbers(unittest.TestCase):
8     def test_mixed_input(self):
9         text = "Hello, World! 123"
10        expected = "hello world"
11        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
12
13    def test_lowercase_only(self):
14        text = "this is a test"
15        expected = "this is a test"
16        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
17
18    def test_punctuation_and_numbers(self):
19        text = "!@#%$^&*()1234567890"
20        expected = ""
21        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
22
23    def test_empty_string(self):
24        text = ""
25        expected = ""
26        self.assertEqual(lowercase_remove_punct_numbers(text), expected)
27
28    def test_spaces_and_special_characters(self):
29        text = " Hello!! World?? "
30        expected = " hello world "
```

```
You are a Python testing assistant.
Given Python code enclosed within {{{ }}} , generate unit tests using the
unittest framework. For each function or method in the code:
1.Create a corresponding test method within a unittest.TestCase subclass.
2.Use meaningful test method names that reflect the function being tested.
3.Include appropriate assertions based on the function's logic and
expected behavior.
4.Use mock objects or patching where necessary (e.g., for I/O, APIs, or
external dependencies).
5.If a function has multiple logical branches or edge cases, include test
cases for them.
6.Do not include the original code in the output-only the test code.Chapter 13
319
7.Import any modules or classes necessary for the tests to run.
8.Format your output as a complete, valid Python test file using the
unittest module.

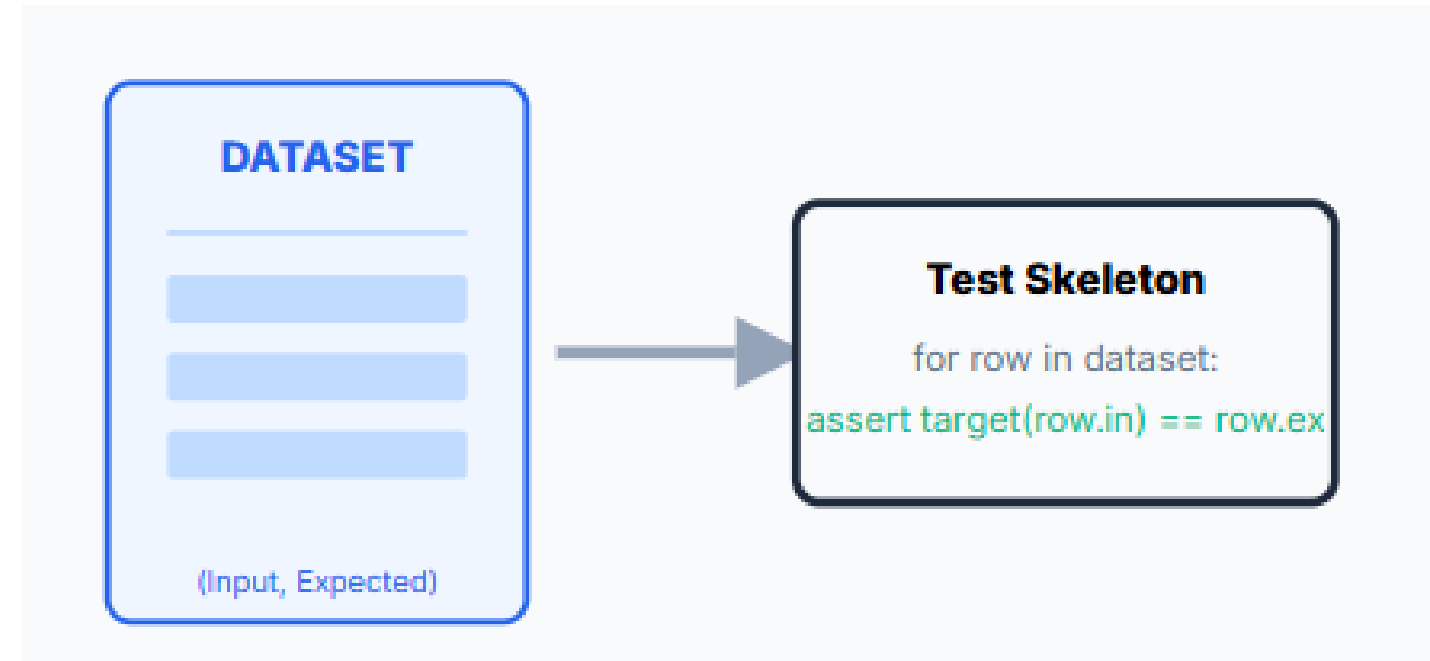
Input:
python
{{{
import re

def lowercase_remove_punct_numbers(text, supercharte=True):
    return re.sub(r'[^a-z\s]', '', text.lower())

def multiple_to_single_spaces(text):
    letters_single_spaces = re.sub(r'\s+', ' ', text)
    return letters_single_spaces
}}}
```

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

3. DATA-DRIVEN TESTING



4. TEST-DRIVEN DEVELOPMENT



BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

5. KẾT HỢP GENAI VỚI TDD

```
CONTEXT: You are provided with
1. Python function enclosed with {{{ FUNCTION }}} that has a
2. Test suite code enclosed with {{{ UNIT TESTS }}}.
3. Error message enclosed with {{{ ERROR }}}.
TASK: Fix the function implementation
FUNCTION: {{{def rect_intersection_area(rect1, rect2):
return NoneWriting and Maintaining Unit Tests
328
}}}}
UNIT TESTS: {{{ ... omitted for brevity ... }}}
ERROR: {{{ ... omitted for brevity ... }}}
CODE: You, 4 days ago • Uncommitted changes
```

```
You, 6 days ago | 1 author (You)
def rect_intersection_area(rect1, rect2): You, 6 days ago
    x1_1, y1_1, x2_1, y2_1 = rect1
    x1_2, y1_2, x2_2, y2_2 = rect2

    # Validate input rectangles
    def is_invalid_rect(x1, y1, x2, y2):
        return x1 >= x2 or y1 >= y2

    if is_invalid_rect(x1_1, y1_1, x2_1, y2_1) or is_invalid_rect(x1_2, y1_2, x2_2, y2_2):
        raise ValueError("Invalid rectangle with non-positive width or height")

    # Find intersection bounds
    inter_left = max(x1_1, x1_2)
    inter_right = min(x2_1, x2_2)
    inter_bottom = max(y1_1, y1_2)
    inter_top = min(y2_1, y2_2)

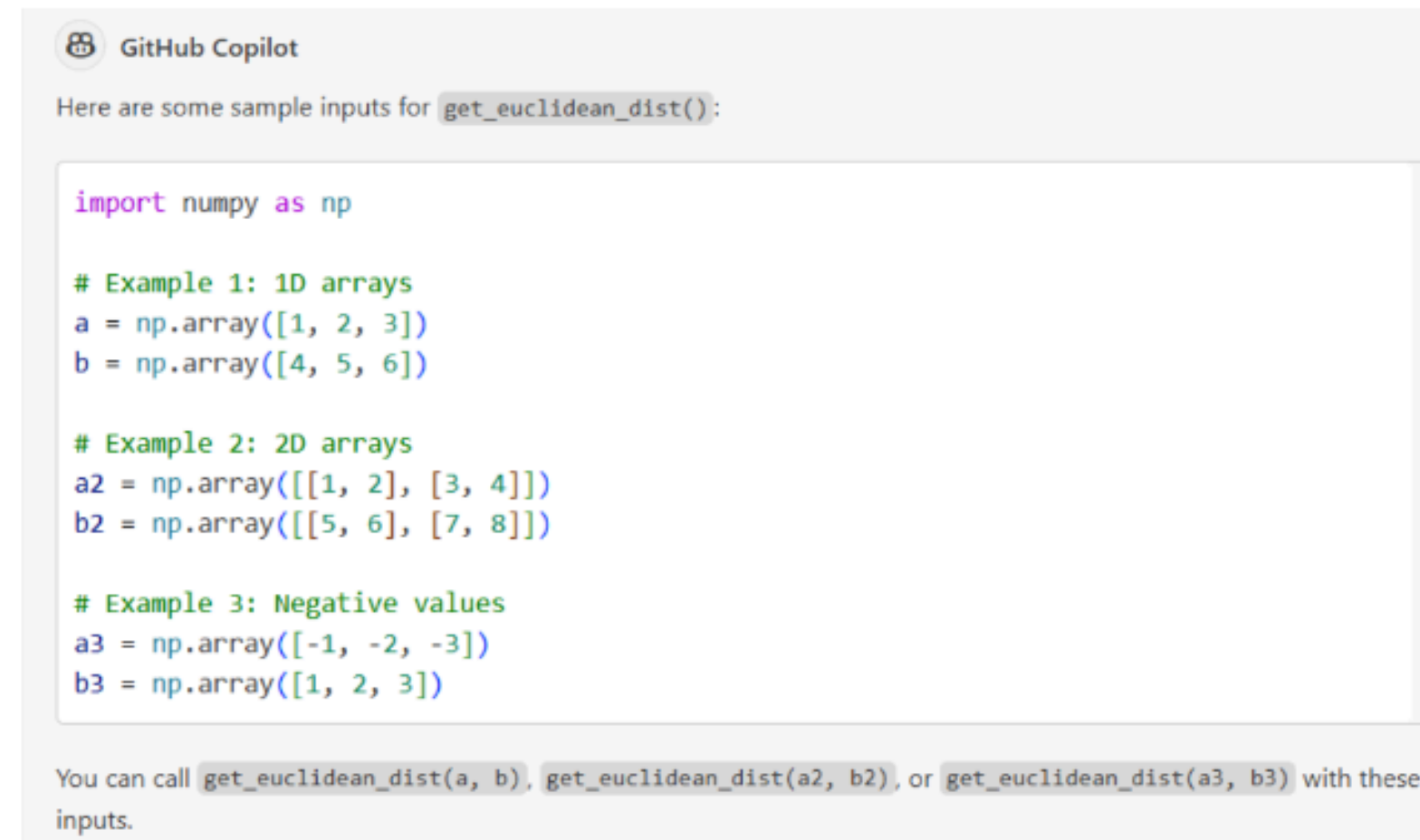
    # Check if rectangles intersect
    if inter_left >= inter_right or inter_bottom >= inter_top:
        return 0

    # Compute and return intersection area
    return (inter_right - inter_left) * (inter_top - inter_bottom)
```

BEHIND THE SCENES: HOW CHATGPT, GITHUB COPILOT, AND OTHER LLMS WORK

6. TẠO DỮ LIỆU KIỂM THỬ BẰNG GENAI

```
def get_euclidean_dist(a: np.ndarray, b: np.ndarray) -> float:
    """Return the squared Euclidean (L2) distance between two arrays."""
    return np.sqrt(np.sum((a - b) ** 2))
```



GENAI FOR RUNTIME AND MEMORY MANAGEMENT

1. PHÂN TÍCH RUNTIME VÀ SPACE COMPLEXITY

RUNTIME CỦA CHƯƠNG TRÌNH

Runtime không phải hằng số, nó phụ thuộc vào:

- Độ phức tạp thuật toán (Big-O).
- Kích thước dữ liệu đầu vào.
- Cấu hình phần cứng.

```
def fibonacci_recursive(n):  
    if n <= 1:  
        return n  
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2)
```

BỘ NHỚ CỦA CHƯƠNG TRÌNH

Space (Độ phức tạp không gian) là chỉ số đo lường lượng bộ nhớ RAM mà chương trình chiếm dụng trong suốt quá trình vận hành

	video_1	video_2	video_3	video_4	video_5	video_6	...
user_1		0.5				1	
user_2	0.1			0.7		0.9	

ĐÁNH ĐỔI: TIME VS MEMORY

Không có thiết kế "tốt nhất" tuyệt đối, chỉ có sự lựa chọn phù hợp:

- Caching
- Chunking

GENAI FOR RUNTIME AND MEMORY MANAGEMENT

2. PROFILING (ĐO HIỆU NĂNG)

Profiling là quá trình đo lường các chỉ số vận hành thực tế của mã nguồn để xác định chính xác các "điểm nghẽn"

GITHUB COPILOT TRONG PROFILING

```
print the runtime of fibonacci_recursive for n=35
```

GitHub Copilot

> 4 steps completed successfully

To print the runtime of `fibonacci_recursive` for `n=35`, you can use the `time` module to measure the start and end times of the function call. Here is the complete code:

```
import time
from fibonacci import fibonacci_recursive

n = 35
start_time = time.time()
result = fibonacci_recursive(n)
end_time = time.time()

print(f"Result: {result}")
print(f"Runtime: {end_time - start_time} seconds")
```

PROFILING RUNTIME

```
import time
from fibonacci import fibonacci_recursive, fibonacci_iterative

n = 35
start_time = time.time()
result = fibonacci_recursive(n)
end_time = time.time()

print(f"Result: {result}")
print(f"Runtime: {end_time - start_time} seconds")
```

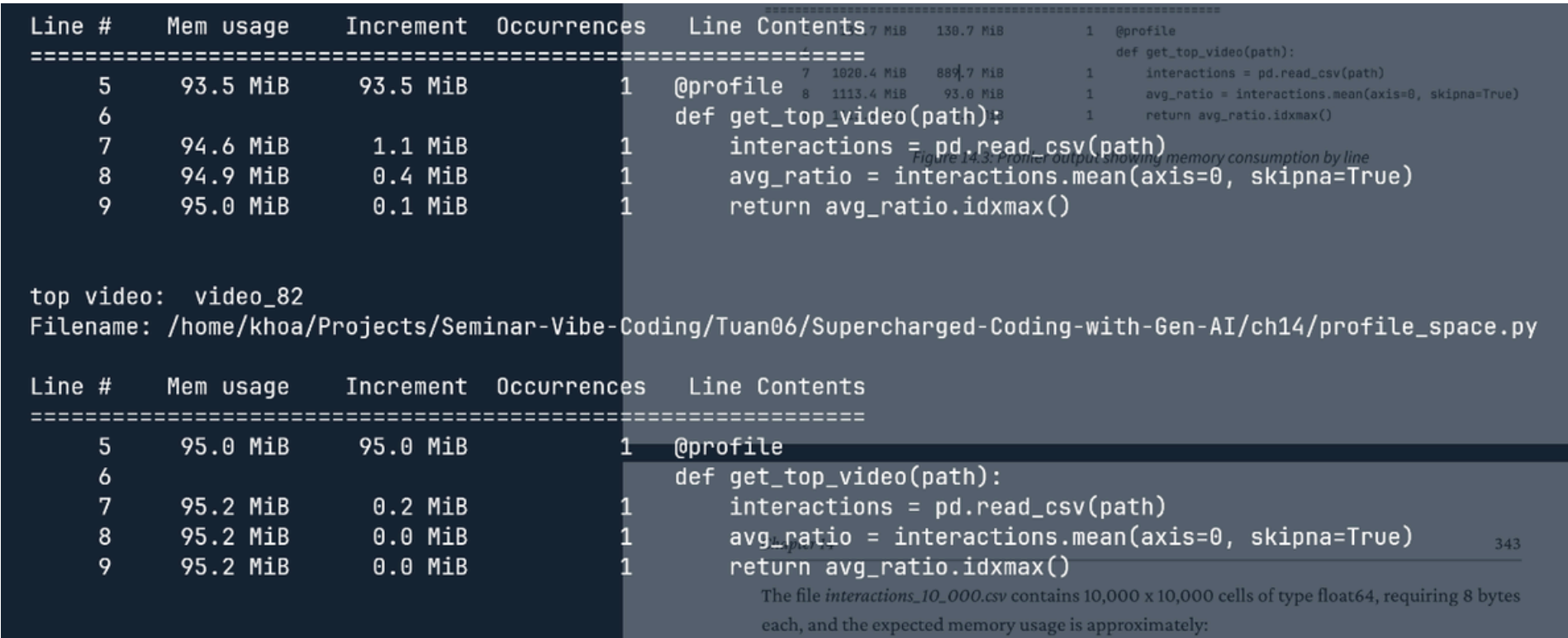
```
Result: 9227465
Runtime: 1.1331148147583008 seconds
```

GENAI FOR RUNTIME AND MEMORY MANAGEMENT

2. PROFILING (ĐO HIỆU NĂNG)

PROFILING MEMORY

```
You, 3 days ago | 1 author (You)
1 import pandas as pd
2 from memory_profiler import profile
3
4
5 @profile
6 def get_top_video(path):
7     interactions = pd.read_csv(path)
8     avg_ratio = interactions.mean(axis=0, skipna=True)
9     return avg_ratio.idxmax()
10
11
12 print("top video: ", get_top_video('interactions_100.csv'))
13
14 # Profiling memory consumption across multiple runs
15 paths = [
16     'interactions_100.csv',
17     # 'interactions_1000.csv',
18     # 'interactions_10_000.csv'
19 ]
20
21 for p in paths:
22     print("top video: ", get_top_video(p))
23
```



GENAI FOR RUNTIME AND MEMORY MANAGEMENT

3. PHÂN TÍCH MAX CAPACITY BẰNG CHATGPT

Max Capacity là ngưỡng giới hạn lớn nhất của dữ liệu đầu vào (input size) mà một chương trình hoặc hệ thống có thể xử lý ổn định trong một khung tài nguyên nhất định

```
Runtime for fibonacci_recursive(n=10:41:5):
decorator_style_guide.py
fibonacci_recursive(10): 0.000008 seconds
fibonacci_recursive(15): 0.000066 seconds
fibonacci_recursive(20): 0.000726 seconds
fibonacci_recursive(25): 0.007778 seconds
```

PHƯƠNG PHÁP:

```
paths = ['interactions_100.csv',
         'interactions_1000.csv',
         'interactions_10_000.csv']

for p in paths:
    print("top video: ", get_top_video(p))
```

```

Filename: /Users/hila/PycharmProjects/private/supercharge/ch14/profile_space.py
Line #    Mem usage    Increment    Occurrences    Line Contents
=====
      5      93.2 MiB      93.2 MiB          1    @profile
      6                                  def get_top_video(path):
      7      93.8 MiB       0.6 MiB          1          interactions = pd.read_csv(path)
      8      94.0 MiB       0.2 MiB          1          avg_ratio = interactions.mean(axis=0, skipna=True)
      9      94.0 MiB       0.0 MiB          1          return avg_ratio.idxmax()

```

top video: video_82

Filename: /Users/hila/PycharmProjects/private/supercharge/ch14/profile_space.py

Line #	Mem usage	Increment	Occurrences	Line Contents
5	94.0 MiB	94.0 MiB	1	@profile
6				def get_top_video(path):
7	124.0 MiB	29.9 MiB	1	interactions = pd.read_csv(path)
8	131.6 MiB	7.6 MiB	1	avg_ratio = interactions.mean(axis=0, skipna=True)
9	131.6 MiB	0.0 MiB	1	return avg_ratio.idxmax()

top video: video_629

Filename: /Users/hila/PycharmProjects/private/supercharge/ch14/profile_space.py

Line #	Mem usage	Increment	Occurrences	Line Contents
5	130.7 MiB	130.7 MiB	1	@profile
6				def get_top_video(path):
7	1020.4 MiB	889.7 MiB	1	interactions = pd.read_csv(path)
8	1113.4 MiB	93.0 MiB	1	avg_ratio = interactions.mean(axis=0, skipna=True)
9	1113.4 MiB	0.0 MiB	1	return avg_ratio.idxmax()

top video: video_7238

GENAI FOR RUNTIME AND MEMORY MANAGEMENT

4. TỐI ƯU CODE BẰNG CHAINED PROMPTS VÀ CÁC HƯỚNG TỐI ƯU NÂNG CAO

Chained Prompt (Chuỗi câu lệnh liên kết) là một kỹ thuật tương tác nâng cao với GenAI, trong đó người dùng không đưa ra yêu cầu cuối cùng ngay lập tức. Thay vào đó, quy trình được chia nhỏ thành một chuỗi các bước logic nối tiếp nhau

TỐI ƯU MEMORY

Prompt 1: SPACE COMPLEXITY

Giải thích:

- Hàm gọi lại chính nó recursively tạo ra call stack
- Cây recursion có độ sâu tối đa = n (con đường dài nhất)
- Mỗi level trong call stack dùng memory → tổng $O(n)$ stack space
- Ví dụ: fib(5) → độ sâu stack ≈ 5

Comparison:

Metric	Recursive	Iterative	Fast Doubling
Time	$O(2^n)$	$O(n)$	$O(\log n)$
Space	$O(n)$	$O(1) \checkmark$	$O(\log n)$
fib(35)	1.7s	0.00001s	0.00001s
fib(100)	× crash	0.0001s	0.00001s
Stack Depth	~100	0 (loop)	$\sim \log(100)=7$

Iterative giảm from $O(n)$ → $O(1)$ space = chỉ dùng 2 biến thay vì lưu cả call stack! 📄

TỐI ƯU RUNTIME

Prompt 1: TIME COMPLEXITY

Giải thích:

- Hàm gọi lại chính nó 2 lần: `fibonacci_recursive(n-1)` + `fibonacci_recursive(n-2)`
- Mỗi call tạo ra 2 branches, mỗi branch lại tạo 2 branches → cây nhị phân
- Tổng số calls: 2^n → Exponential complexity

Comparison:

n	Recursive	Iterative	Fast Doubling
35	1.7s	~0.00001s	~0.00001s
40	19.5s	~0.00001s	~0.00001s
100	× crash	~0.00005s	~0.00001s
100,000	× impossible	~0.05s	~0.00001s

Fast Doubling giảm từ $O(2^n)$ → $O(\log n)$ = hơn 1 triệu lần nhanh hơn! ⚡

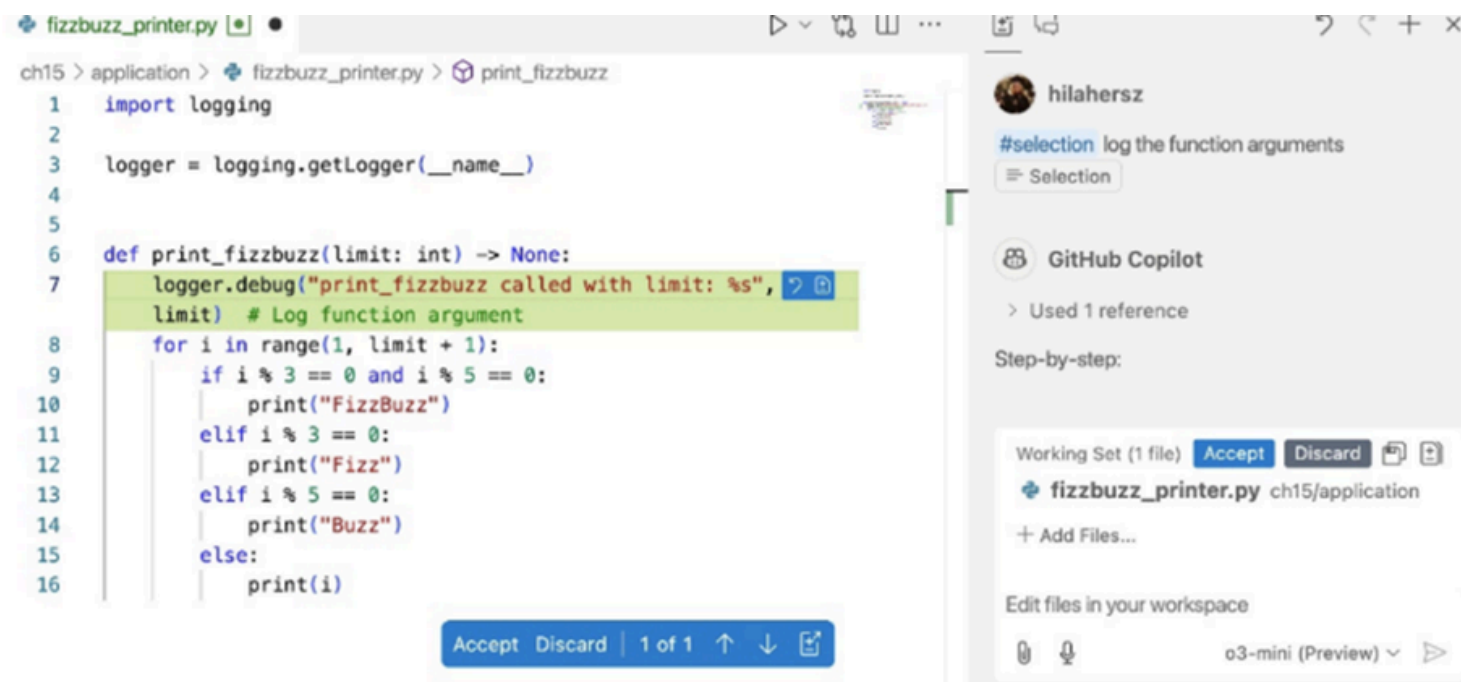
CÁC HƯỚNG TỐI ƯU NÂNG CAO

- XỬ LÝ SONG SONG
- TÍNH TOÁN TRÊN GPU
- MA TRẬN THỪA
- ĐỊNH DẠNG DỮ LIỆU TỐI ƯU

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

1. LOGGING, MONITORING VÀ CƠ CHẾ PHÁT SINH LỖI

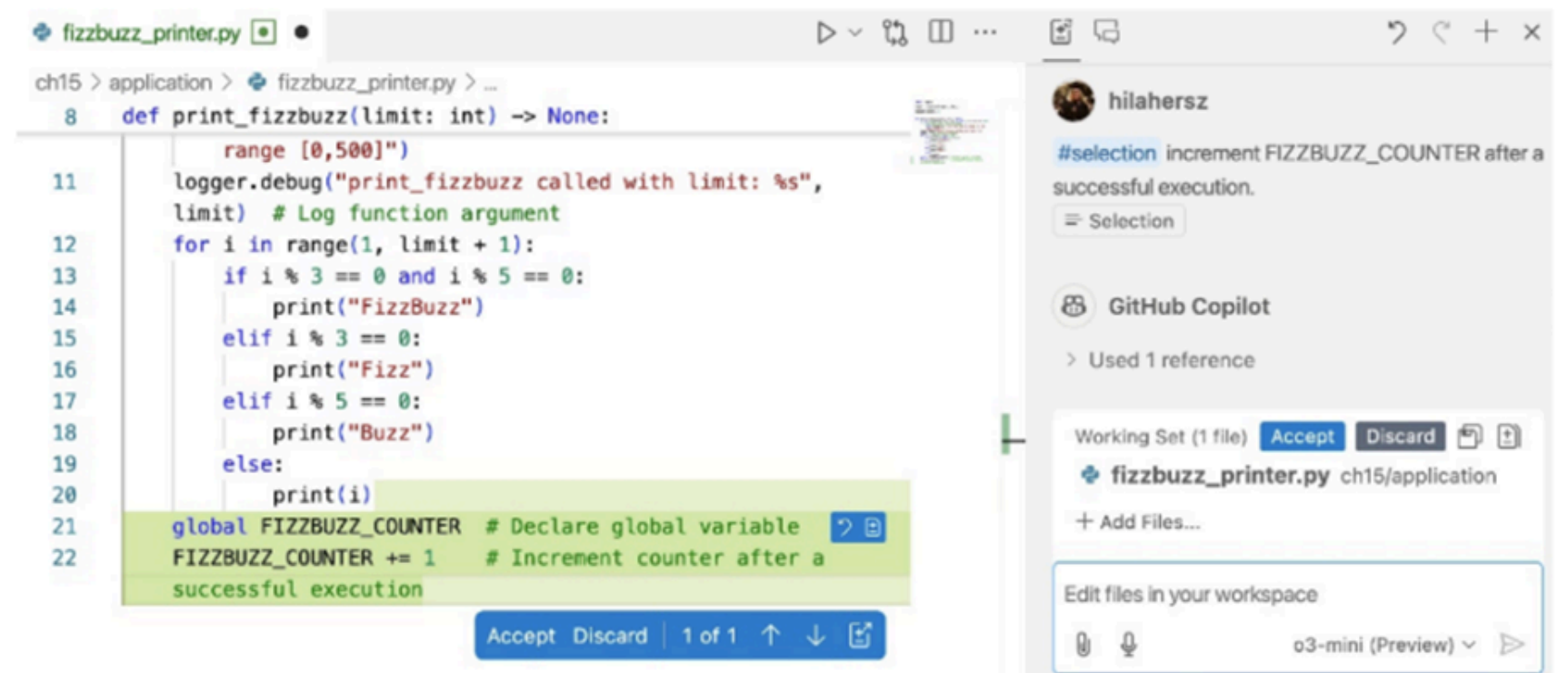
LOGGING



```
fizzbuzz_printer.py [x] •
ch15 > application > fizzbuzz_printer.py > print_fizzbuzz
1 import logging
2
3 logger = logging.getLogger(__name__)
4
5
6 def print_fizzbuzz(limit: int) -> None:
7     logger.debug("print_fizzbuzz called with limit: %s",
8                 limit) # Log function argument
9     for i in range(1, limit + 1):
10         if i % 3 == 0 and i % 5 == 0:
11             print("FizzBuzz")
12         elif i % 3 == 0:
13             print("Fizz")
14         elif i % 5 == 0:
15             print("Buzz")
16         else:
17             print(i)
```

#selection increment FIZZBUZZ_COUNTER after a successful execution

with the o3 mini model:



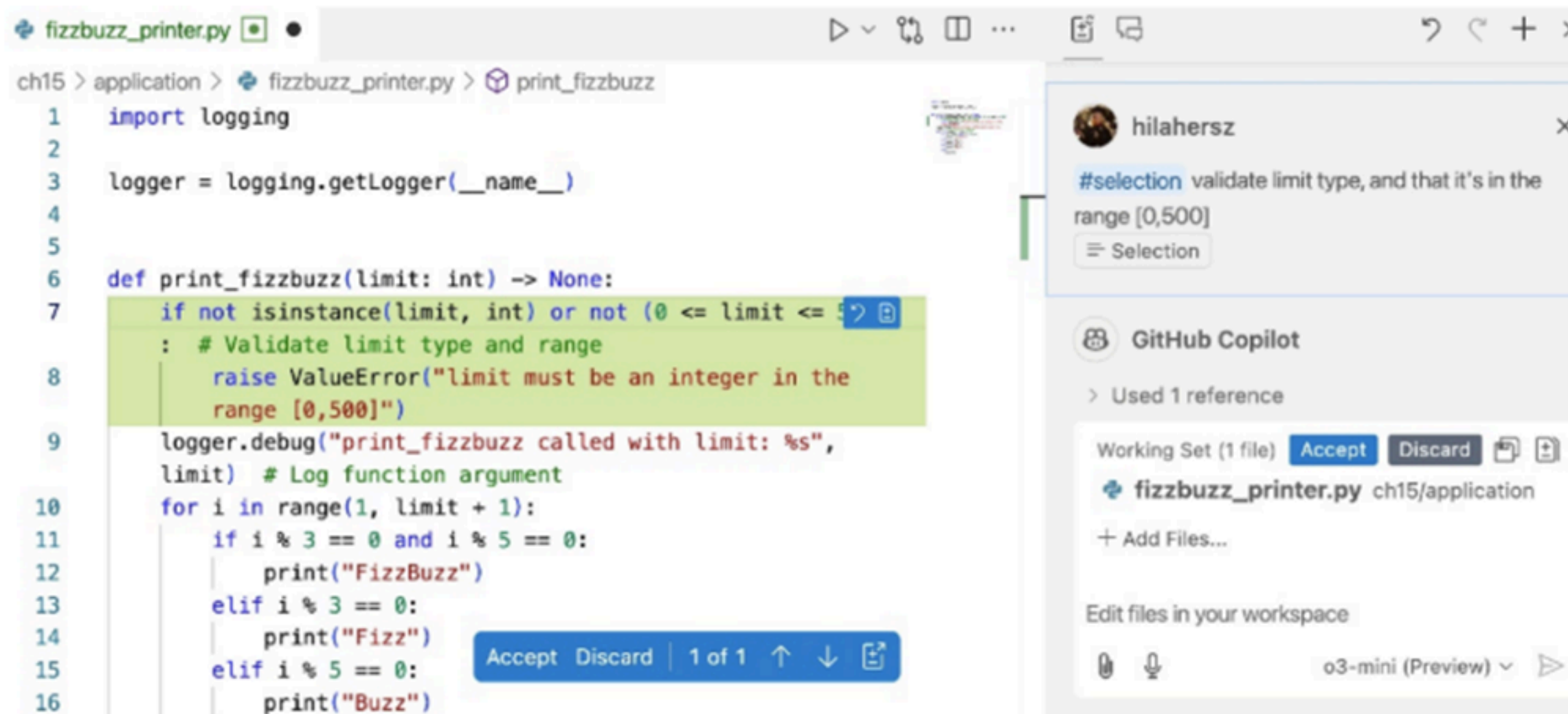
```
fizzbuzz_printer.py [x] •
ch15 > application > fizzbuzz_printer.py > ...
8 def print_fizzbuzz(limit: int) -> None:
9     range [0,500]")
10
11     logger.debug("print_fizzbuzz called with limit: %s",
12                 limit) # Log function argument
13     for i in range(1, limit + 1):
14         if i % 3 == 0 and i % 5 == 0:
15             print("FizzBuzz")
16         elif i % 3 == 0:
17             print("Fizz")
18         elif i % 5 == 0:
19             print("Buzz")
20         else:
21             print(i)
22
23     global FIZZBUZZ_COUNTER # Declare global variable
24     FIZZBUZZ_COUNTER += 1 # Increment counter after a
25     successful execution
```

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

1. LOGGING, MONITORING VÀ CƠ CHẾ PHÁT SINH LỖI

RAISING ERRORS

#selection validate limit type, and that it is in the range [0,500]



The screenshot shows a code editor with a file named `fizzbuzz_printer.py`. The code is as follows:

```
1 import logging
2
3 logger = logging.getLogger(__name__)
4
5
6 def print_fizzbuzz(limit: int) -> None:
7     if not isinstance(limit, int) or not (0 <= limit <= 500):
8         : # Validate limit type and range
9         raise ValueError("limit must be an integer in the
10         range [0,500]")
11
12     logger.debug("print_fizzbuzz called with limit: %s",
13     limit) # Log function argument
14     for i in range(1, limit + 1):
15         if i % 3 == 0 and i % 5 == 0:
16             print("FizzBuzz")
17         elif i % 3 == 0:
18             print("Fizz")
19         elif i % 5 == 0:
20             print("Buzz")
```

The sidebar on the right shows the GitHub Copilot interface. It displays the selected code snippet: `#selection validate limit type, and that it's in the range [0,500]`. Below this, it shows the file `fizzbuzz_printer.py` in the workspace `ch15/application`. At the bottom, it indicates the model used is `o3-mini (Preview)`.

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

2. ỨNG DỤNG GENAI TRONG VIỆC XÂY DỰNG CÁC MÔ HÌNH THIẾT KẾ CODE Ở MỨC CAO

DECORATOR PATTERN

Thay vì nhúng trực tiếp các chức năng phụ vào trong function, ta sử dụng design pattern để tách riêng chúng.

```
def sample_decorator(func: callable) -> callable:
    def wrapper(*args, **kwargs):
        print("Function is wrapped")
        return func(*args, **kwargs)

    return wrapper
```

```
@sample_decorator
def foo(*num):
    return len(nums)
```

TRIỂN KHAI COT NGƯỢC VỚI DECORATOR

```
get_geometric_mean.py x
1  from typing import Union
2  import cmath
3
4
5  def get_geometric_mean(*args: float) -> Union[float, complex]: new *
6      product = get_product(*args)
7      nth_root = len(args)
8      is_positive = is_product_positive(product)
9      is_nth_root_odd = is_odd(nth_root)
10     is_mean_real = is_geometric_mean_real(is_nth_root_odd, is_positive)
11     mean = get_mean(product, nth_root, is_mean_real)
12
13     return mean
```

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

2. ỨNG DỤNG GENAI TRONG VIỆC XÂY DỰNG CÁC MÔ HÌNH THIẾT KẾ CODE Ở MỨC CAO TRIỂN KHAI COT NGƯỢC VỚI DECORATOR

```
fizzbuzz_printer.py x
1 import logging
2
3 logger = logging.getLogger(__name__)
4
5 FIZZBUZZ_COUNTER = 0
6
7
8 @log_function_args new *
9 @increment_counter
10 @validate_args_types_and_limits(0, 500)
11 def print_fizzbuzz(limit: int) -> None:
```

```
def 1

fizzbuzz_printer.py x
1 import logging
2
3 logger = logging.getLogger(__name__)
4
5 FIZZBUZZ_COUNTER = 0
6
7
8 def log_function_args(func): new *
    def wrapper(*args, **kwargs):
        logger.info(f"Function {func.__name__} called with args: {args}, kwargs: {kwargs}")
        return func(*args, **kwargs)
    return wrapper
9
10
11 @log_function_args new *
12 @increment_counter
13 @validate_args_types_and_limits(0, 500)
14 def print_fizzbuzz(limit: int) -> None:
15     for i in range(1, limit + 1):
```

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

3. ỨNG DỤNG COT NGƯỢC VỚI CHATGPT VÀ OPENAI

CHAT GPT

CONTEXT: You are provided with a partial Python script in {{{FIZZBUZZ_PRINTER}}}, where some of the code is missing
TASK: Implement the decorators with the missing implementation in the code with the same style of the project files

```
FIZZBUZZ_PRINTER: {{{
import logging

logger = logging.getLogger(__name__)

FIZZBUZZ_COUNTER = 0

@log_function_args
@increment_counter
@validate_args_types_and_limits(0, 500)
def print_fizzbuzz(limit: int) -> None:
    for i in range(1, limit + 1):
        if i % 3 == 0 and i % 5 == 0:
            print("FizzBuzz")
        elif i % 3 == 0:
            print("Fizz")
        elif i % 5 == 0:
            print("Buzz")
        else:
            print(i)
}}}
```

CODE:

```
def log_function_args(func: callable) -> callable:
    def wrapper(*args, **kwargs):
        logger.info(
            f"Calling function '{func.__name__}' with args: {args} and
kwargs: {kwargs}"
        )
        return func(*args, **kwargs)
    return wrapper

def increment_counter(func: callable) -> callable:
    def wrapper(*args, **kwargs):
        global FIZZBUZZ_COUNTER
        FIZZBUZZ_COUNTER += 1
        logger.info(f"FIZZBUZZ_COUNTER incremented to {FIZZBUZZ_COUNTER}")
        return func(*args, **kwargs)
    return wrapper

def validate_args_types_and_limits(min_limit: int, max_limit: int):
    def decorator(func: callable) -> callable:
        def wrapper(*args, **kwargs):
            # Use inspect to bind the arguments to the function's
signature
            sig = inspect.signature(func)
            bound_args = sig.bind(*args, **kwargs)
            bound_args.apply_defaults()
            # Expecting the argument to be named 'limit'
```

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

3. ỨNG DỤNG COT NGƯỢC VỚI CHATGPT VÀ OPENAI

OPEN API

VÍ DỤ SYSTEM PROMPT

```
SURROUND = "You are provided with a partial Python script in {{{FIZZBUZZ_PRINTER}}}, where some of the code is missing  
SINGLE_TASK = "Implement the decorators with the missing implementation in the code."
```

VÍ DỤ USER PROMPT

```
def get_user_prompt(script_path: str) -> str:  
    with open(script_path, 'r') as file:  
        incomplete_code = file.read()  
  
    return f"""  
    FIZZBUZZ_PRINTER: {{{{{{ {incomplete_code} }}}}}}  
  
    CODE:  
    """
```

hilahersz, 7 months ago • ch15: Going live; logging, monitoring and error...

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

4. SỬ DỤNG FEW-SHOT LEARNING VÀ FINE-TUNING LÀM STYLE GUIDE

FEW-SHOT LEARNING VỚI GITHUB COPILOT

```
import logging
import time
from functools import wraps
from typing import Any

logger: logging.Logger = logging.getLogger(__name__)

def time_it(func: callable) -> callable:
    @wraps(func)
    def wrapper(*args, **kwargs):
        start_time: float = time.time()
        res: Any = func(*args, **kwargs)
        end_time: float = time.time()
        logger.info(
            "Function called.",
            extra={
                "function": func.__name__,
                "args": args,
                "kwargs": kwargs,
                "error": "",
                "timing": f"{end_time - start_time} seconds"})
        return res
    return wrapper

@time_it
def my_func(a: int, b: int) -> int:
    return a + b
```

```
def log_function_args(func: callable) -> callable: new *
    def wrapper(*args, **kwargs):
        message: str = "Function called."
        logger.info(message,
                    extra={"function": func.__name__,
                          "args": args,
                          "kwargs": kwargs,
                          "error": "",
                          "timing": ""})
        return func(*args, **kwargs)
    return wrapper

@log_function_args 2 usages new *
@increment_counter
@validate_args_types_and_limits(0, 500)
def print_fizzbuzz(limit: int) -> None:
```

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

4. SỬ DỤNG FEW-SHOT LEARNING VÀ FINE-TUNING LÀM STYLE GUIDE

FEW-SHOT LEARNING VỚI CHATGPT

CONTEXT: You are provided with a partial Python script enclosed with {{{FIZZBUZZ_PRINTER}}} where some of the code is missing, and examples of a good implementation enclosed with {{{ EXAMPLES }}}
TASK: Implement the decorators with the missing implementation in the code while following the style guide.

EXAMPLES:

INCOMPLETE_CODE: {{{...omitted for brevity...}}}

COMPLETE_CODE: {{{...omitted for brevity...}}}

FIZZBUZZ_PRINTER: {{{...omitted for brevity...}}}

CODE:

ini-high

```
def log_function_args(func: Callable) -> Callable:
    @wraps(func)
    def wrapper(*args: Any, **kwargs: Any) -> Any:
        logger.info(
            "Function called.",
            extra={
                "function": func.__name__,
                "args": args,
                "kwargs": kwargs,
                "error": ""
            }
        )
        return func(*args, **kwargs)
    return wrapper
```

Copy

GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

4. SỬ DỤNG FEW-SHOT LEARNING VÀ FINE-TUNING LÀM STYLE GUIDE

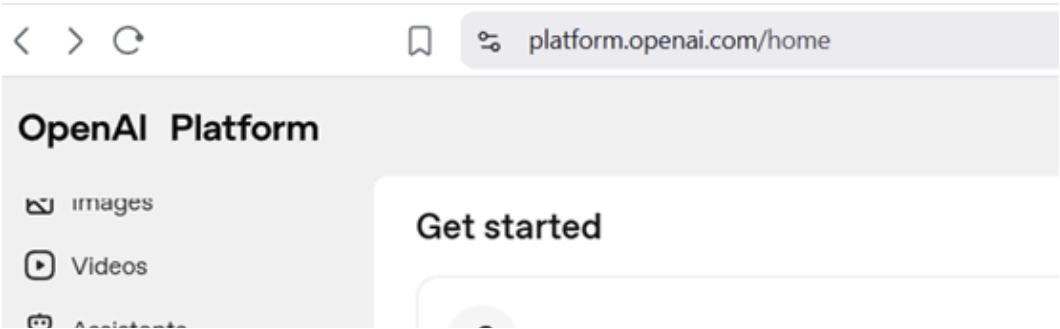
FINE-TUNING VỚI OPENAI API

```
{
  "messages": [
    {
      "role": "system",
      "content": "You will be provided with a Python function signature enclosed with {{{ FUNCTION }}}. Your task is to implement it."
    },
    {
      "role": "user",
      "content": "FUNCTION: {{{def get_arithmetic_mean(a, b)}}}\n CODE: "
    },
    {
      "role": "assistant",
      "content": "def get_arithmetic_mean(a, b): \n    return (a+b)/2", "weight": 1
    }
  ]
}
```

KẾT QUẢ

MODEL	
ft:gpt-4o-mini-2024-07-18:pazpaz-the-coder:decorator-style:AydMPYCg	
Status	Succeeded
Job ID	ftjob-zDa73ZywRRDInwOHngNeAfUy
Training Method	Supervised
Suffix	decorator-style
Base model	gpt-4o-mini-2024-07-18
Output model	ft:gpt-4o-mini-2024-07-18:pazpaz-the-coder:decorator-style:AydMPYCg
Created at	Feb 8, 2025, 1:24 PM

QUY TRÌNH



Create a fine-tuned model

Method
Specify the method to be used for fine-tuning.

Supervised

Base Model

gpt-4o-mini-2024-07-18

Suffix
Add a custom suffix that will be appended to the output model name.

decorator-style

Seed
The seed controls the reproducibility of the job. Passing in the same seed and job parameters should produce the same results, but may differ in rare cases. If a seed is not specified, one will be generated for you.

Random

Training data
Add a jsonl file to use for training. By providing the file, you confirm that you have the rights to use the data.

☐ Upload new

☒ Select existing

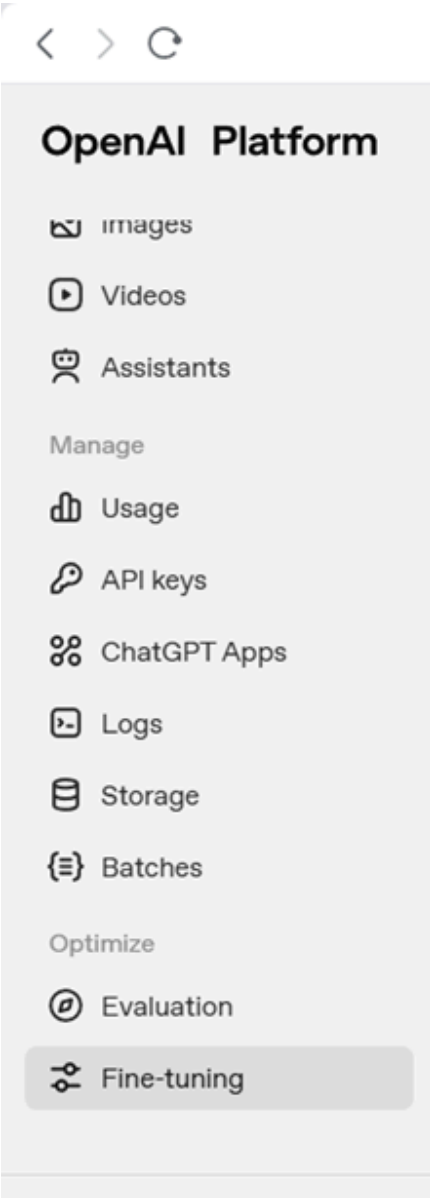
Browse files

file-FxKnmvzkV8FwQjYd589V

Learn about fine-tuning

Cancel

Create



GOING LIVE WITH GENAI: LOGGING, MONITORING, AND ERRORS

4. SỬ DỤNG FEW-SHOT LEARNING VÀ FINE-TUNING LÀM STYLE GUIDE

SO SÁNH

VÍ DỤ SYSTEM PROMPT

You are provided with a partial Python script in {{{FIZZBUZZ_PRINTER}}}, where some of the code is missing. Your task is to implement the decorators with the missing implementation in the code.

VÍ DỤ USER PROMPT

FIZZBUZZ_PRINTER: {{{ ...omitted for brevity... }}}
CODE :

KẾT QUẢ

Chat

gpt-4o-mini-2024-07-18

...

...

×

Assistant

To complete your FizzBuzz printer implementation with the missing decorators, we can define three decorators: 'log_function_args', 'increment_counter', and 'validate_args_types_and_limits'. Below is the implementation of these decorators alongside the existing code you've provided.

```
python
import logging

logger = logging.getLogger(__name__)

FIZZBUZZ_COUNTER = 0

def log_function_args(func):
    def wrapper(*args, **kwargs):
        logger.info(f'Function {func.__name__} called with arguments: {args} and keyword arguments: {kwargs}')
        return func(*args, **kwargs)
    return wrapper

def increment_counter(func):
```

11,554ms

+ 222t

+ 612t

Request ID

ft:gpt-4o-mini-2024-07-18:pazpar-the-coder:decorator-style:AydMPYcG

...

...

×

FIZZBUZZ_PRINTER: {{{

import logging\n\nlogging.basicConfig(level=logging.INFO)\n\nfilename='fizzbuzz.log'\n\nlogger = logging.getLogger(__name__)\n\nFIZZBUZZ_COUNTER = 0\n\n@log_function_args\n@increment_counter\n@validate_args_types_and_limits(0, 500)\ndef print_fizzbuzz(limit: int) -> None:\n for i in range(1, limit + 1):\n if i % 3 == 0 and i % 5 == 0:\n print('FizzBuzz')\n elif i % 3 == 0:\n print('Fizz')\n elif i % 5 == 0:\n print('Buzz')\n else:\n print(i)\n\n}}}

CODE:

Assistant

import logging\n\nfrom functools import wraps\n\nlogger = logging.getLogger(__name__)\n\nFIZZBUZZ_COUNTER = 0\n\ndef log_function_args(func):\n @wraps(func)\n def wrapper(*args, **kwargs):\n logger.info(f'Function called:',\n extra={

5,438ms

+ 222t

+ 123t

Request ID

ARCHITECTURE, DESIGN, AND THE FUTURE

1. SỰ PHÁT TRIỂN NHANH CHÓNG CỦA GENAI VÀ TÁC ĐỘNG ĐẾN KINH TẾ PHẦN MỀM

01

SỰ PHÁT TRIỂN NHANH CHÓNG CỦA GENAI

- Tối ưu hóa chi phí sản xuất
- Rút ngắn thời gian đưa sản phẩm ra thị trường
- Thay đổi cán cân bảo trì
- Dân chủ hóa phát triển phần mềm

02

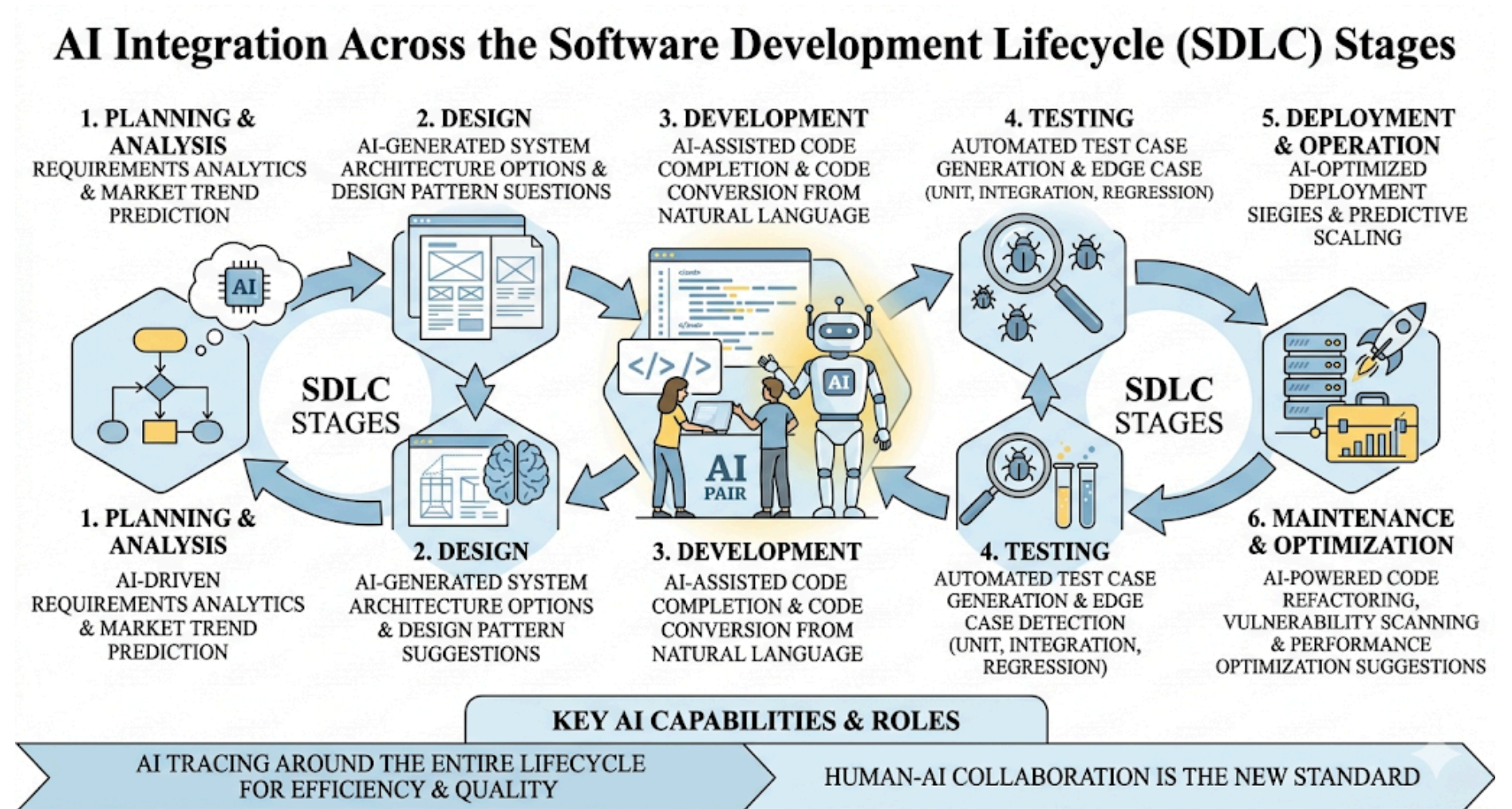
TÁC ĐỘNG ĐẾN KINH TẾ PHẦN MỀM

- Tăng trưởng năng suất đột phá
- Tối ưu hóa chi phí đầu tư
- Sự bùng nổ về cung phần mềm
- Kích cầu thị trường công nghệ

ARCHITECTURE, DESIGN, AND THE FUTURE

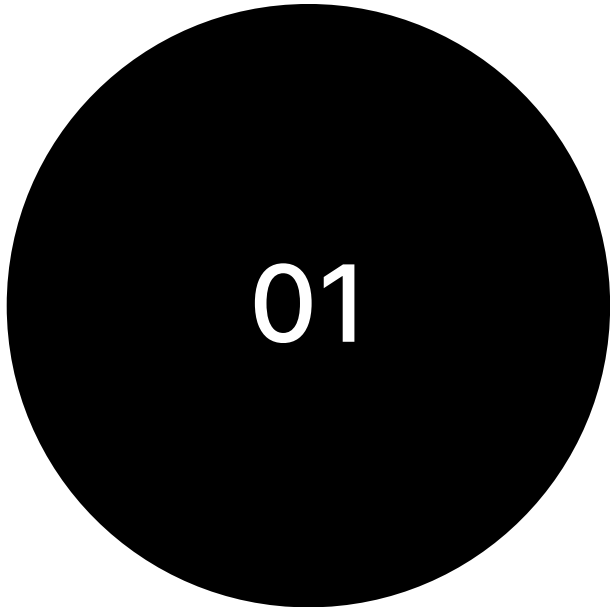
2. MỨC ĐỘ CHẤP NHẬN CỦA GENAI

- Viết mã nguồn (Coding):
- Tạo kịch bản kiểm thử (Test Case Generation)
- Sinh tài liệu kỹ thuật (Documentation)
- Debug và tối ưu hóa



ARCHITECTURE, DESIGN, AND THE FUTURE

3. THAY ĐỔI TRONG VAI TRÒ CỦA LẬP TRÌNH VIÊN



01

LẬP TRÌNH VIÊN MỚI (JUNIOR DEVELOPER)

- Tập trung vào các nhiệm vụ thực thi
- Hỗ trợ học tập và tăng tốc
- Nguy cơ thiếu hụt chiều sâu



02

LẬP TRÌNH VIÊN CẤP CAO VÀ CHUYÊN GIA (SENIOR DEVELOPER)

- Tập trung vào thiết kế và kiến trúc
- Giải quyết các bài toán phức tạp và đặc thù
- Vai trò Thẩm định và Kiểm soát

ARCHITECTURE, DESIGN, AND THE FUTURE

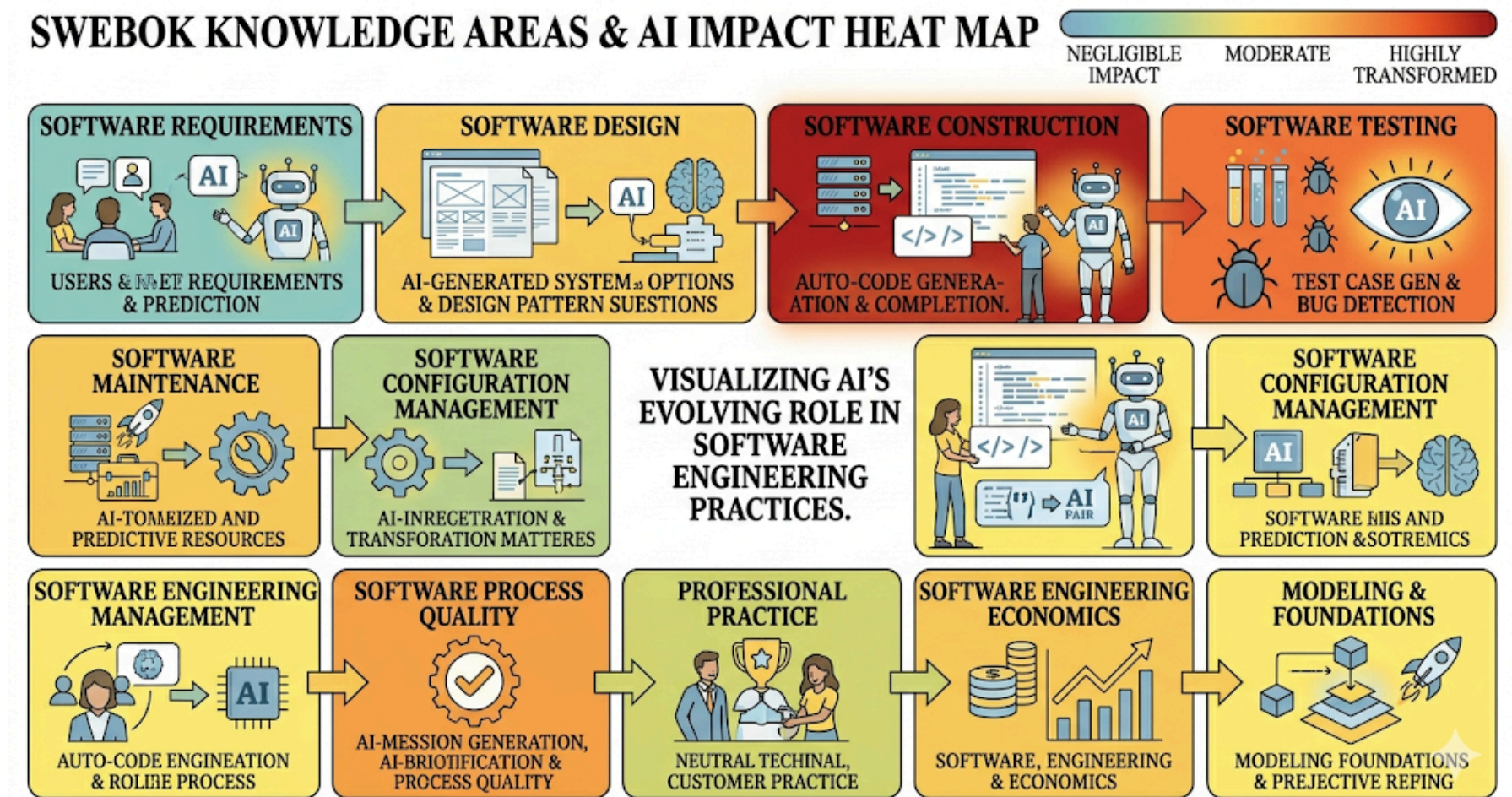
4. GENAI VÀ SWEBOK

CÁC LĨNH VỰC CHỊU TÁC ĐỘNG MẠNH (AI-DOMINANT)

Xây dựng phần mềm (Software Construction) và Kiểm thử phần mềm (Software Testing)

CÁC LĨNH VỰC VẪN DO CON NGƯỜI DẪN DẮT (HUMAN-CENTRIC)

- Kiến trúc phần mềm (Software Architecture)
- Thiết kế phần mềm (Software Design)
- Quản lý kỹ thuật phần mềm (Software Management)



ARCHITECTURE, DESIGN, AND THE FUTURE

5. DÂN CHỦ HÓA LẬP TRÌNH

Xóa nhòa ranh giới giữa lập trình viên và người dùng cuối. Ngôn ngữ tự nhiên trở thành giao diện mới, cho phép chuyên gia phi kỹ thuật tự xây dựng giải pháp phần mềm chuyên biệt.

6. ẢNH HƯỞNG ĐẾN HỆ THỐNG LEGACY VÀ TỔ CHỨC

01

HỒI SINH CÁC HỆ THỐNG LEGACY

- Hiểu và bảo trì hệ thống cũ
- Tái cấu trúc và viết lại code
- Giảm sự phụ thuộc vào kỹ năng hiếm

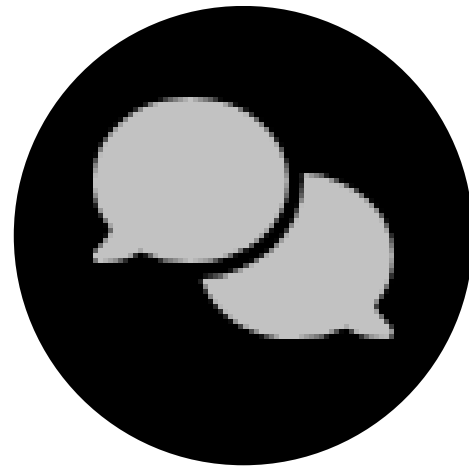
02

TĂNG CƯỜNG NĂNG LỰC VÀ TÍNH LINH HOẠT CỦA TỔ CHỨC

- Tăng tính linh hoạt của đội ngũ phát triển
- Cho phép chuyển đổi giữa các dự án nhanh hơn

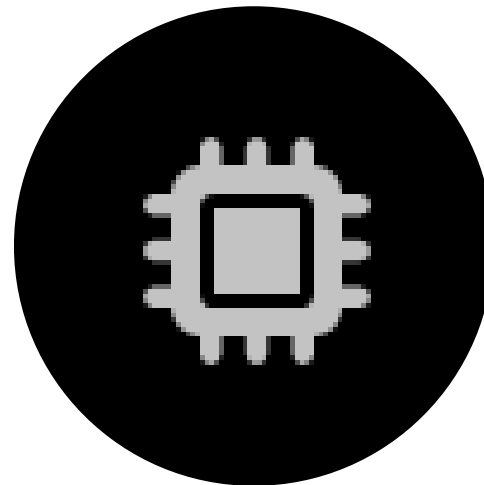
ARCHITECTURE, DESIGN, AND THE FUTURE

7. TƯƠNG LAI CỦA NGÔN NGỮ LẬP TRÌNH



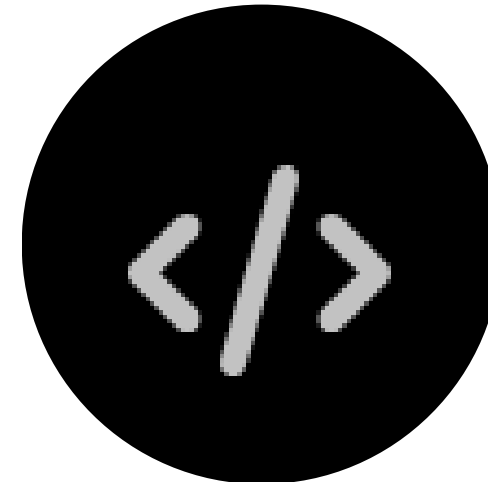
NGÔN NGỮ TỰ NHIÊN

sẽ trở thành ngôn ngữ lập trình chính, mô tả yêu cầu "hệ thống cần làm gì".



COMPILER THÔNG MINH

AI đóng vai trò trình biên dịch, chuyển mô tả thành mã thực thi, tối ưu hóa bảo mật và hiệu năng.



PSEUDOCODE

Giải pháp trung gian kết hợp tư duy logic của con người và khả năng xử lý của máy tính.

ARCHITECTURE, DESIGN, AND THE FUTURE

8. VIBE CODING

Truyền tải ý tưởng thay vì cú pháp

- Con người là nhà đạo diễn: Phác thảo "vibe" và luồng tư duy.
- AI là người thực thi: Tự động suy luận công nghệ và hiện thực hóa ý tưởng.
- Tạo mẫu cực nhanh: Lý tưởng cho Startup và Prototyping trong vài phút.

9. TƯƠNG LAI CỦA SOFTWARE ENGINEERING

- AI tích hợp sâu vào các IDE (Môi trường phát triển tích hợp)
- Tăng trưởng năng suất làm việc (Productivity Surge)
- Cải thiện chất lượng code (Code Quality Improvement)
- Tự động chuyển đổi từ Requirement sang Code (End-to-End Automation)
- Tích hợp Đa phương thức
- Vai trò "Tổng công trình sư":



ARCHITECTURE, DESIGN, AND THE FUTURE

LIỆU AI CÓ THAY THẾ LẬP TRÌNH VIÊN?

0%

AI thiếu Tư duy phê phán và khả năng đánh đổi (Trade-offs) trong bối cảnh kinh doanh thực tế.

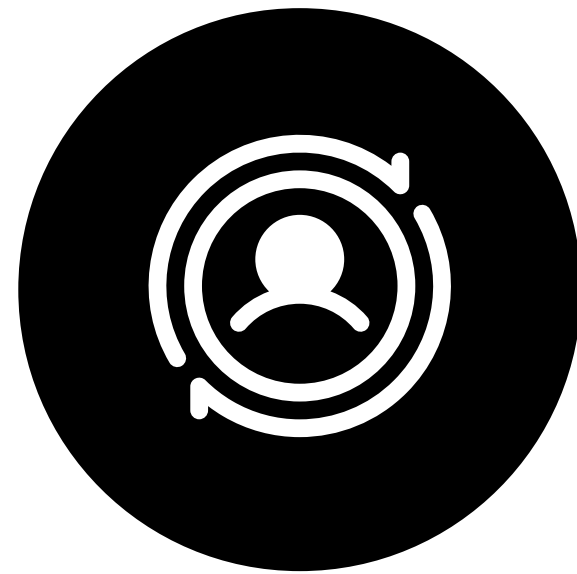
- AI chỉ giỏi ở các nhiệm vụ nhỏ và cục bộ
- Sự phụ thuộc vào dữ liệu huấn luyện
- Tư duy phê phán và Đánh đổi (Trade-offs)

RỦI RO VÀ QUẢN TRỊ

Thách thức cốt lõi	Chi tiết rủi ro
Trách nhiệm pháp lý	Ai chịu trách nhiệm khi mã AI gây rò rỉ dữ liệu hoặc lỗi hệ thống nghiêm trọng?
Bản quyền (IP)	Nguy cơ vi phạm giấy phép Open Source (GPL, MIT) từ dữ liệu huấn luyện.
Tính minh bạch	Vấn đề "Hộp đen" - Khó giải thích logic phức tạp do AI tự động tạo ra.
Tuân thủ (Compliance)	Các lĩnh vực nhạy cảm (FinTech, HealthTech) yêu cầu kiểm soát dữ liệu nội bộ chặt chẽ.

ARCHITECTURE, DESIGN, AND THE FUTURE

TÁC ĐỘNG ĐẾN GIÁO DỤC VÀ TUYỂN DỤNG



SỰ DỊCH CHUYỂN TRONG GIÁO DỤC ĐÀO TẠO

- Kỹ năng viết code thủ công (Coding Syntax) giảm tầm quan trọng
- Trọng tâm mới - Tư duy hệ thống và Giải quyết vấn đề



CÁCH MẠNG TRONG TUYỂN DỤNG

- Thay đổi các bài toán thuật toán (LeetCode-style)
- Đánh giá khả năng "Hợp tác với AI"
- Phỏng vấn dựa trên hệ thống (Architecture-based Interview)

**THANK YOU FOR
YOUR ATTENTION**

